

Home Automation Sign Language Recognition

with application of MediaPipe Using Random Forest Algorithm

Complete User Guide & Setup Instructions

This document provides step-by-step instructions for installing, training, and running the real-time ASL Home Automation system described in the thesis project.

Project	ASL Home Automation Control Center
Technologies	MediaPipe Hands + Random Forest + Arduino Uno R3
Gestures	5 static American Sign Language (ASL) commands
Hardware	Smartphone camera, Laptop, Arduino + 5V Relay Module
Mode	Fully offline / local processing (zero cloud latency)

1. Project Overview

The system enables Deaf and Hard-of-Hearing (DHH) users to control household appliances using American Sign Language (ASL) hand gestures. A smartphone camera captures the hand, MediaPipe extracts 21 three-dimensional landmarks (63 numeric features), a Random Forest classifier identifies the gesture, and an Arduino Uno R3 drives relay modules to switch appliances on or off.

System Pipeline

Smartphone (DroidCam) → USB → Laptop → MediaPipe Hands → Random Forest → USB Serial → Arduino Uno R3 → 5V Relay → Appliances

Supported Gestures

Gesture	Serial Cmd	Action
LIGHTS_ON	1	Living Room Light ON
LIGHTS_OFF	2	Living Room Light OFF
FAN_ON	3	Bedroom Fan ON
FAN_OFF	4	Bedroom Fan OFF
AUX	5	Toggle Aux Appliance

2. Hardware Requirements

- Laptop or desktop PC (Windows / Linux / macOS)
- Smartphone with DroidCam (or any USB webcam)
- Arduino Uno R3 microcontroller
- 5V Relay Module (4-channel or 8-channel recommended)
- Jumper wires (male-to-female)
- Household appliances (lights, fan, etc.) connected safely through relays
- USB cable for Arduino and (optionally) for smartphone

Wiring Diagram (Arduino ↔ Relay Module)

Arduino Pin	Relay Channel	Appliance
D2	Relay 1	Light 1 (Living Room)
D3	Relay 2	Fan 1 (Bedroom)
D4	Relay 3	Aux Appliance
D5	Relay 4	(Optional)
D6	Relay 5	(Optional)
5V	VCC	Power
GND	GND	Ground

Safety Warning: Always use the relay module to switch high-voltage AC loads. Never connect household appliances directly to the Arduino pins.

3. Software Installation

Step 1 — Create a virtual environment (recommended)

```
python -m venv venv  
source venv/bin/activate # Linux / macOS  
venv\Scripts\activate # Windows
```

Step 2 — Install Python dependencies

```
pip install -r requirements.txt
```

Required packages:

opencv-python, mediapipe, scikit-learn, joblib, pyserial, numpy, Pillow, customtkinter

4. Collect Training Data

Before the system can recognize gestures, you must collect landmark samples for each of the five ASL commands.

```
python train_model.py
```

Then choose option **1** (Collect data).

Controls during collection:

- Press **0 – 4** to select which gesture you are recording
- Hold the hand pose steady in front of the camera
- Press **s** to save the current buffer of samples
- Press **c** to clear the buffer without saving
- Press **q** to quit

Recommendation: Collect at least 100–200 samples per gesture under consistent lighting for good accuracy. More samples generally improve Random Forest performance.

5. Train the Random Forest Model

```
python train_model.py
```

Then choose option **2** (Train model).

The script loads all saved .npy files from the `gesture_data/` folder, splits the data into train/test sets (80/20), trains a Random Forest classifier, prints a classification report, and saves the model as `asl_rf_model.joblib`.

6. Upload Arduino Firmware

1. Open `arduino_relay.ino` in the Arduino IDE.
2. Select the correct board: **Tools** → **Board** → **Arduino Uno**.
3. Select the correct COM port: **Tools** → **Port**.
4. Click the Upload button.

Once uploaded, the Arduino will listen on Serial at 9600 baud and switch the corresponding relay pin when it receives the characters '1' through '5'.

7. Run the Main Application

Before launching, edit these two lines near the top of **main_app.py**:

```
SERIAL_PORT = "COM3" # e.g. "COM3" on Windows or "/dev/ttyUSB0" on Linux  
CAMERA_INDEX = 0 # Try 1 or 2 if using DroidCam
```

Then start the application:

```
python main_app.py
```

UI Features

- Live video feed with MediaPipe landmarks drawn on the hand
- Real-time detected gesture name and confidence score
- Adjustable minimum confidence cutoff slider
- Live status of each connected appliance relay (ON / OFF)
- Serial & system log panel with timestamps
- Fully offline operation (no internet required)

8. Project Structure

ASL_Home_Automation/

```
■■■ main_app.py      # Main real-time application (CustomTkinter UI)  
■■■ train_model.py   # Data collection + Random Forest training  
■■■ arduino_relay.ino # Arduino firmware for the relays  
■■■ requirements.txt # Python dependencies  
■■■ asl_rf_model.joblib # Generated after training  
■■■ gesture_data/    # Saved landmark samples (.npy)  
■■■ README.md        # Quick reference
```

9. Notes for Thesis Evaluation

The system is designed to support evaluation against the ISO/IEC 25010 software quality model as stated in the research objectives:

- **Functional Suitability** — Correct recognition of the five defined ASL gestures and proper appliance switching
- **Performance Efficiency** — Real-time landmark extraction and classification with low latency
- **Interaction Capability** — Intuitive visual feedback and confidence threshold control
- **Reliability** — Debounce logic and confidence filtering to reduce false triggers
- **Security** — Local-only processing; no cloud transmission of video or commands

Limitations (as defined in the thesis scope): only five static gestures are supported; the system is intended for controlled indoor environments; continuous ASL sentence translation and multi-hand recognition are outside the current scope.